

Student Reference

OpenCV and NumPy Used in the Drone Vision Lab

This reference explains the OpenCV methods, OpenCV constants, NumPy methods, NumPy properties, and NumPy data structures used in the drone classroom exercises. The goal is to help you understand what each tool does, why it appears in the code, and how it helps with drone-video perception, telemetry, HUD drawing, and control.

Context	Prerecorded drone videos used as a simulated onboard camera
Main uses	Video input, image display, HUD overlays, color detection, brightness measurement, image-based control
Libraries	OpenCV for image/video operations, NumPy for arrays and measurements
Audience	Student

How to read this document

- What it is: the purpose of the method, property, constant, or data type.
- Why it is used: the reason it appears in the drone lab.
- Example: a short code fragment similar to the classroom code.
- Useful in this context: how it helps with the drone-video activities.

OpenCV Reference

cv2.VideoCapture(path)

Type: Method / constructor

What it is: Opens a video file so frames can be read one by one.

Why it is used: It is the starting point for treating a prerecorded drone video as a simulated onboard camera.

Useful in this context: Used in the camera, teleop, telemetry, target-tracking, and integration challenges to load the drone footage.

Example

```
cap = cv2.VideoCapture(str(VIDEO_PATH))
```

cap.isOpened()

Type: Method

What it is: Checks whether the video source opened successfully.

Why it is used: Prevents confusing errors later in the loop when students try to read from an invalid capture object.

Useful in this context: Useful whenever the teacher changes the video folder or a filename is wrong.

Example

```
if not cap.isOpened():  
    print('Could not open video')
```

cap.read()

Type: Method

What it is: Reads the next frame from the video and returns two values: a success flag and the image.

Why it is used: It drives the frame-processing loop.

Useful in this context: This is the heartbeat of the whole lab, because every perception and control step begins from the current frame.

Example

```
ok, frame = cap.read()  
if not ok:  
    break
```

cap.get(cv2.CAP_PROP_FPS)

Type: Method + property constant

What it is: Retrieves the frames-per-second value stored in the video metadata.

Why it is used: Lets us estimate time using `frame_index / fps`.

Useful in this context: Important in telemetry alignment and when turning frame number into a timestamp.

Example

```
fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
```

cap.release()

Type: Method

What it is: Closes the video file and releases the capture resource.

Why it is used: Good cleanup prevents locked files and inconsistent behavior.

Useful in this context: Always call it at the end of a challenge script.

Example

```
cap.release()
```

cv2.resize(image, dsize=None, fx=..., fy=...)

Type: Method

What it is: Resizes an image to a different scale.

Why it is used: Drone videos can be 4K, which is too large for comfortable classroom processing.

Useful in this context: Makes the exercises faster, keeps windows manageable, and reduces computation.

Example

```
frame = cv2.resize(frame, dsize=None, fx=0.4, fy=0.4)
```

cv2.putText(...)

Type: Method

What it is: Draws text directly on an image.

Why it is used: Lets students build HUD overlays for time, brightness, commands, telemetry, and target area.

Useful in this context: Vital for turning raw processing into interpretable output during class.

Example

```
cv2.putText(frame, 'frame: 10', (20, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

cv2.FONT_HERSHEY_SIMPLEX

Type: Constant

What it is: A built-in OpenCV font used when drawing text.

Why it is used: Required by putText to choose a text style.

Useful in this context: Keeps on-screen information readable with a minimal setup.

Example

```
cv2.putText(frame, text, pos, cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
```

cv2.imshow(window_name, image)

Type: Method

What it is: Displays an image in a named window.

Why it is used: Allows students to see the current camera frame, target mask, and HUD.

Useful in this context: Makes the lab interactive and visually testable.

Example

```
cv2.imshow('challenge_05', frame)
```

cv2.waitKey(delay_ms)

Type: Method

What it is: Waits briefly for a keyboard press and keeps the window responsive.

Why it is used: Needed both for playback timing and for keyboard control.

Useful in this context: Used for quitting with q and for teleoperation keys.

Example

```
key = cv2.waitKey(20) & 0xFF
```

cv2.destroyAllWindows()

Type: Method

What it is: Closes all OpenCV windows created by the script.

Why it is used: Completes cleanup at the end of a program.

Useful in this context: Prevents stray windows between challenge runs.

Example

```
cv2.destroyAllWindows()
```

cv2.cvtColor(image, code)

Type: Method

What it is: Converts an image from one color representation to another.

Why it is used: Used when moving from BGR camera images into grayscale or HSV for analysis.

Useful in this context: Central to brightness measurement, color thresholding, and mask display.

Example

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

cv2.COLOR_BGR2GRAY

Type: Constant

What it is: Conversion code from a color image to grayscale.

Why it is used: Needed before computing brightness with one intensity channel.

Useful in this context: Simplifies measurements when color is not needed.

Example

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

cv2.COLOR_BGR2HSV

Type: Constant

What it is: Conversion code from BGR to HSV color space.

Why it is used: HSV is often easier for color-based segmentation than BGR.

Useful in this context: Used for red target detection in the perception challenges.

Example

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

cv2.COLOR_GRAY2BGR

Type: Constant

What it is: Conversion code from a grayscale image into a 3-channel BGR image.

Why it is used: Lets a mask be displayed inside a color frame overlay.

Useful in this context: Helpful when showing a mask inset on top of the main camera image.

Example

```
mask_bgr = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)
```

cv2.inRange(image, lower, upper)

Type: Method

What it is: Creates a binary mask where pixels inside the chosen range become white and the rest become black.

Why it is used: This is the main segmentation step for the virtual red target.

Useful in this context: Turns a color problem into a simple binary detection problem.

Example

```
mask = cv2.inRange(hsv, lower, upper)
```

cv2.bitwise_or(mask1, mask2)

Type: Method

What it is: Combines two binary masks into one.

Why it is used: Red often wraps around the HSV hue scale, so two ranges are combined.

Useful in this context: Improves red target detection robustness.

Example

```
mask = cv2.bitwise_or(mask1, mask2)
```

cv2.circle(image, center, radius, color, thickness)

Type: Method

What it is: Draws a circle on the image.

Why it is used: Used both to create the virtual target and to draw the detected centroid marker.

Useful in this context: Lets students both inject a target and visualize a detection result.

Example

```
cv2.circle(frame, (cx, cy), 18, (0, 0, 255), -1)
```

cv2.line(image, pt1, pt2, color, thickness)

Type: Method

What it is: Draws a line between two points.

Why it is used: Used to draw crosshairs in the HUD.

Useful in this context: Helps students visually compare target position to the desired image center.

Example

```
cv2.line(frame, (w // 2, 0), (w // 2, h), (255, 255, 255), 1)
```

NumPy Reference

np.ndarray

Type: Array type / core object

What it is: The main NumPy array type used to store images and masks.

Why it is used: OpenCV frames are represented as NumPy arrays, so every image operation depends on understanding this object.

Useful in this context: Images, masks, and overlays all live as arrays in memory.

Example

```
image: np.ndarray
```

array.shape

Type: Property

What it is: Returns the size of an array along each dimension.

Why it is used: For images, shape helps us find height, width, and sometimes channels.

Useful in this context: Used to place targets, draw crosshairs, compute image center, and build inset displays.

Example

```
h, w = frame.shape[:2]
```

np.array([...], dtype=np.uint8)

Type: Method

What it is: Creates a NumPy array from a list of values.

Why it is used: Used to define HSV lower and upper threshold bounds.

Useful in this context: Makes threshold values compatible with OpenCV image operations.

Example

```
lower = np.array([0, 140, 120], dtype=np.uint8)
```

np.uint8

Type: Data type

What it is: Unsigned 8-bit integer type with values from 0 to 255.

Why it is used: Standard image channel values are usually stored in this type.

Useful in this context: Important for masks, color thresholds, and image buffers.

Example

```
np.array([255, 0, 0], dtype=np.uint8)
```

np.mean(array)

Type: Method

What it is: Computes the average value of the array.

Why it is used: Used to estimate frame brightness from grayscale images.

Useful in this context: Shows how a full image can be reduced to a single measurement.

Example

```
brightness = float(np.mean(gray))
```

np.where(condition)

Type: Method

What it is: Returns the positions where a condition is true.

Why it is used: Used to find all white pixels in a mask after thresholding.

Useful in this context: This is how the lab moves from a binary mask to a target centroid and area.

Example

```
ys, xs = np.where(mask > 0)
```

np.zeros(shape, dtype=np.uint8)

Type: Method

What it is: Creates an array filled with zeros.

Why it is used: Used as a placeholder mask in starter files.

Useful in this context: Good for initializing blank grayscale images and masks.

Example

```
mask = np.zeros(frame.shape[:2], dtype=np.uint8)
```

Array slicing and assignment

Type: Property / syntax pattern

What it is: Selects and updates a rectangular region inside an array.

Why it is used: Used to place the mask inset inside the main frame.

Useful in this context: Shows students that overlays can be done by writing directly into an image array.

Example

```
frame[y0:y1, x0:x1] = mask_bgr
```

Worked Context Examples

Example 1: Reading a drone video like a camera stream

```
cap = cv2.VideoCapture(str(VIDEO_PATH))
fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
frame_index = 0
ok, frame = cap.read()
timestamp_s = frame_index / fps
```

This example combines VideoCapture, CAP_PROP_FPS, read, and a timestamp computation. It shows how prerecorded video becomes a time-based sensor stream.

Example 2: Measuring brightness

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
brightness = float(np.mean(gray))
```

This example shows how cvtColor and np.mean reduce a full image to a simple numeric measurement.

Example 3: Detecting the virtual red target

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask1 = cv2.inRange(hsv, np.array([0, 140, 120], dtype=np.uint8), np.array([10, 255, 255], dtype=np.uint8))
mask2 = cv2.inRange(hsv, np.array([170, 140, 120], dtype=np.uint8), np.array([180, 255, 255], dtype=np.uint8))
mask = cv2.bitwise_or(mask1, mask2)
ys, xs = np.where(mask > 0)
```

This example shows how OpenCV and NumPy work together: OpenCV builds the mask, and NumPy extracts the pixel positions.

Example 4: Drawing a HUD

```
h, w = frame.shape[:2]
cv2.line(frame, (w // 2, 0), (w // 2, h), (255, 255, 255), 1)
cv2.circle(frame, (int(cx), int(cy)), 20, (0, 255, 255), 2)
cv2.putText(frame, 'mode: TRACK', (20, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (50, 255, 50), 2)
```

This example shows how image dimensions, line drawing, circles, and text create an explainable operator display.